

GO

Functions in Go

Types and Structure — A Clear, Practical Overview

BY EN.Abouelfetouh Moussa

```
func main() {  
  
    functions.Explore()  
  
}
```

Function Types at a Glance

Go organizes functions into ten distinct forms, based on how they are defined and used.

1 Standard Functions

2 Multiple Return Values

3 Named Return Values

4 Anonymous Functions

5 First-Class Functions

6 Closures

7 Recursive Functions

8 Variadic Functions

9 Methods (Struct Functions)

10 Special Functions (main / init)

1 Standard Functions

- Traditional functions with a specific, fixed name.
- Defined using the `func` keyword.
- The most common and basic function form in Go.

```
func multiply(a int, b int) int {  
    return a * b  
}
```

2 Multiple Return Values

- A built-in Go feature: a function can return more than one value.
- Most commonly used to return a result together with an error.
- Removes the need for exceptions to signal failure.

```
func divide(a, b float64) (float64, error) {  
    if b == 0 {  
        return 0, errors.New(  
            "cannot divide by zero")  
        }  
    return a / b, nil  
}
```

3 Named Return Values

- Return variables can be named in advance in the function signature.
- A bare return automatically sends back the named values.
- Makes intent clearer for functions with simple outputs.

```
func getRectangleArea(width, height int) (area  
int) {  
    area = width * height  
    return // returns area automatically  
}
```

4 Anonymous Functions

- Functions defined without a name.
- Often assigned to a variable or invoked immediately.
- Useful for short, one-off pieces of logic.

```
// Immediate execution
func() {
    fmt.Println("Hello Anonymous!")
}()
```

5 First-Class Functions

- Functions are treated like any other data type in Go.
- They can be passed as parameters to other functions.
- They can also be returned as values from a function.

```
func compute(  
    operation func(int, int) int,  
    a, b int,  
) int {  
    return operation(a, b)  
}
```

6 Closures

- An anonymous function bound to variables outside its own scope.
- It retains those values even after the outer function has finished.
- Commonly used to create counters and generators.

```
func sequenceGenerator() func() int {  
    i := 0  
    return func() int {  
        i++  
        return i  
    }  
}
```

7

Recursive Functions

- Functions that call themselves to solve repetitive problems.
- A classic example is calculating a factorial.
- Requires a clear base case to stop the recursion.

```
func factorial(n uint) uint {  
    if n == 0 {  
        return 1  
    }  
    return n * factorial(n-1)  
}
```

8 Variadic Functions

- Accept an unspecified number of arguments.
- Declared using ... before the parameter's type.
- The arguments are available inside as a slice.

```
func sum(numbers ...int) int {  
    total := 0  
    for _, num := range numbers {  
        total += num  
    }  
    return total  
}
```

9 Methods (Struct Functions)

- Functions associated with a specific type, usually a struct.
- The association is made through a receiver.
- This is how Go attaches behavior to data.

```
type Circle struct {  
    Radius float64  
}  
  
// Method attached to Circle  
func (c Circle) Area() float64 {  
    return 3.14 * c.Radius * c.Radius  
}
```

10 Special Functions

- `main()` — the entry point of every Go program, found in package `main`.
- `init()` — runs automatically before `main()`, used to set up configuration or initial variables.
- Both are reserved by the Go runtime and require no explicit call.

```
func main() {  
    // Entry point of the program  
}  
  
func init() {  
    // Runs automatically before main()  
}
```

Thank You

—